

Atividade 3.1

A diretoria comercial pediu um indicador rápido do comportamento de compra. Calcule, apenas para pedidos aprovados, a quantidade de pedidos, o ticket médio (arredondado em 2 casas), o menor e o maior valor.

The screenshot displays a database management interface with two query windows. The left sidebar shows the 'ecommerce_nexashop' schema with tables: 'avaliacoes', 'clientes', 'pedidos', and 'produtos'. The top query window, titled 'Query 1 x pedidos', contains the following SQL code:

```
36 • describe pedidos;
37
38 /*TAREFA 3.2 FATURAMENTO POR FORMA DE PAGAMENTO*/
39
40 • SELECT SUM(valor_total) AS faturamento_aprov
41 FROM pedidos
42 WHERE status= 'aprovado';
43
44 • SELECT forma_pagamento, COUNT(*) AS total_aprovados
```

The result grid for the first query shows the following data:

faturamento_aprov
14003654.52

The bottom query window contains the following SQL code:

```
21 FROM pedidos
22 WHERE status = 'aprovado';
23
24 • SELECT ROUND(AVG(valor_total), 2) AS media_valor_aprovados
25 FROM pedidos
26 WHERE status = 'aprovado';
27
28 • SELECT MIN(valor_total) AS valor_min
29 FROM pedidos
```

The result grid for the second query shows the following data:

media_valor_aprovados
1657.63

SCHEMAS

Filter objects

- ecommerce_nexashop
 - Tables
 - avaliacoes
 - clientes
 - pedidos
 - produtos
 - Views
 - Stored Procedures
 - Functions
- sys

```

21 FROM pedidos
22 WHERE status = 'aprovado';
23
24 • SELECT ROUND(AVG(valor_total), 2) AS media_valor_aprovados
25 FROM pedidos
26 WHERE status = 'aprovado';
27
28 • SELECT MIN(valor_total) AS valor_min
29 FROM pedidos

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

media_valor_aprovados
1657.63

Navigator

SCHEMAS

Filter objects

- ecommerce_nexashop
 - Tables
 - avaliacoes
 - clientes
 - pedidos
 - produtos
 - Views
 - Stored Procedures
 - Functions
- sys

Query 1 x pedidos

```

30 WHERE status= 'aprovado';
31
32 • SELECT MAX(valor_total) AS valor_min
33 FROM pedidos
34 WHERE status= 'aprovado';
35
36 • describe pedidos;
37

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

valor_min
22356.60

Atividade 3.2

Calcule o faturamento total (SUM) de pedidos aprovados, agrupado por forma de pagamento, do maior para o menor.

```

40 • SELECT forma_pagamento, SUM(valor_total) AS faturamento_aprov
41 FROM pedidos
42 WHERE status = 'aprovado'
43 GROUP BY forma_pagamento
44 ORDER BY faturamento_aprov ASC;
45

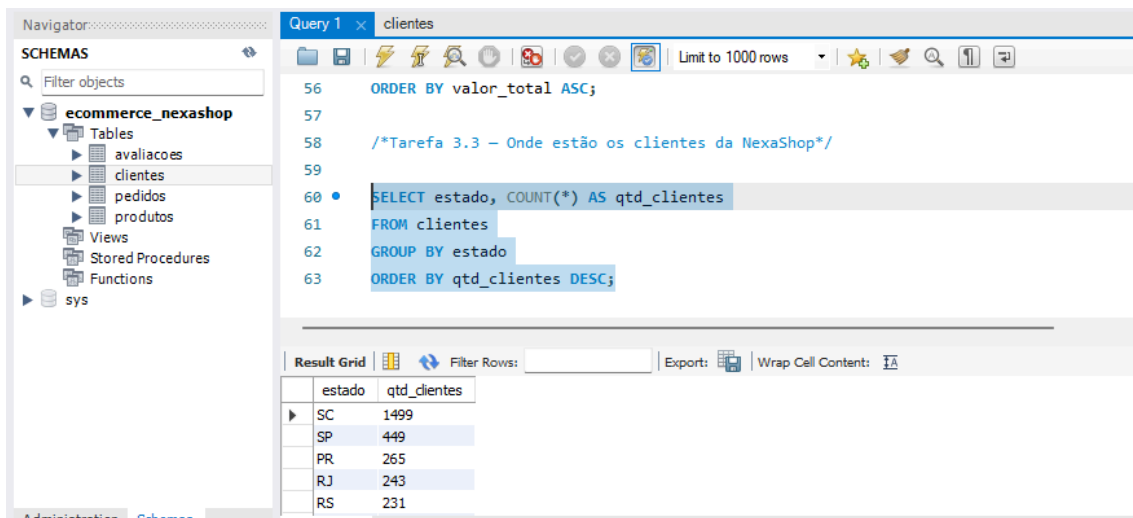
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

forma_pagamento	faturamento_aprov
Cartão de Débito	1196485.65
Boleto	1299030.75
Pix	5212400.88
Cartão de Crédito	6295737.24

Atividade 3.3

Mostre a quantidade de clientes por estado, ordenando do estado com mais clientes para o com menos.



The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'SCHEMAS' pane displays the 'ecommerce_nexashop' database structure, including tables like 'avaliacoes', 'clientes', 'pedidos', and 'produtos'. The main query window, titled 'Query 1 - clientes', contains the following SQL code:

```
56 ORDER BY valor_total ASC;
57
58 /*Tarefa 3.3 – Onde estão os clientes da NexaShop*/
59
60 • SELECT estado, COUNT(*) AS qtd_clientes
61 FROM clientes
62 GROUP BY estado
63 ORDER BY qtd_clientes DESC;
```

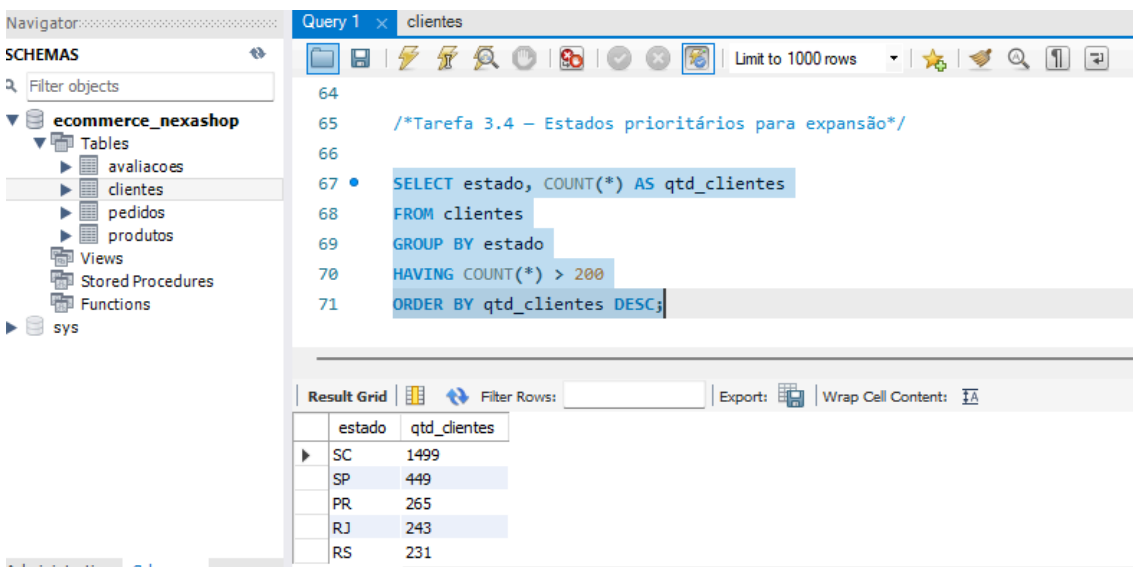
Below the query editor, the 'Result Grid' shows the results of the query:

estado	qtd_clientes
SC	1499
SP	449
PR	265
RJ	243
RS	231

Atividade 3.4

Contexto: O time de expansão só quer investir em estados com presença relevante de clientes.

Tarefa: Liste apenas os estados com mais de 200 clientes cadastrados. Evidência esperada: Uso correto de HAVING (não confundir com WHERE)



The screenshot shows the SQL Server Enterprise Manager interface. The 'SCHEMAS' pane on the left is identical to the previous screenshot. The main query window, titled 'Query 1 - clientes', contains the following SQL code:

```
64
65 /*Tarefa 3.4 – Estados prioritários para expansão*/
66
67 • SELECT estado, COUNT(*) AS qtd_clientes
68 FROM clientes
69 GROUP BY estado
70 HAVING COUNT(*) > 200
71 ORDER BY qtd_clientes DESC;
```

Below the query editor, the 'Result Grid' shows the results of the query, which are identical to the previous screenshot:

estado	qtd_clientes
SC	1499
SP	449
PR	265
RJ	243
RS	231

Atividade 3.5

Tarefa 3.5 — Perfil etário por segmento de cliente Tarefa: Calcule a idade média dos clientes (usando TIMESTAMPDIFF) agrupada por segmento (Varejo, Atacado, Corporativo).

Evidência esperada: Função de data combinada corretamente com GROUP BY.

The screenshot shows a database management tool interface. On the left, the 'Navigator' pane displays the 'ecommerce_nexashop' schema with tables 'avaliacoes', 'clientes', 'pedidos', and 'produtos'. The 'clientes' table is selected. The main pane shows a SQL query titled 'Query 1' with the following code:

```
65 /*Tarefa 3.4 - Estados prioritários para expansão*/
66
67 SELECT estado, COUNT(*) AS qtd_clientes
68 FROM clientes
69 GROUP BY estado
70 HAVING COUNT(*) > 200
71 ORDER BY qtd_clientes DESC;
72
73 SELECT
74     nome,
75     TIMESTAMPDIFF(YEAR, data_nascimento, CURDATE()) AS idade
76 FROM clientes;
77
78 SELECT segmento,
79     ROUND(AVG(TIMESTAMPDIFF(YEAR, data_nascimento, CURDATE())), 2) AS media_idade
80 FROM clientes
81 GROUP BY segmento;
82
```

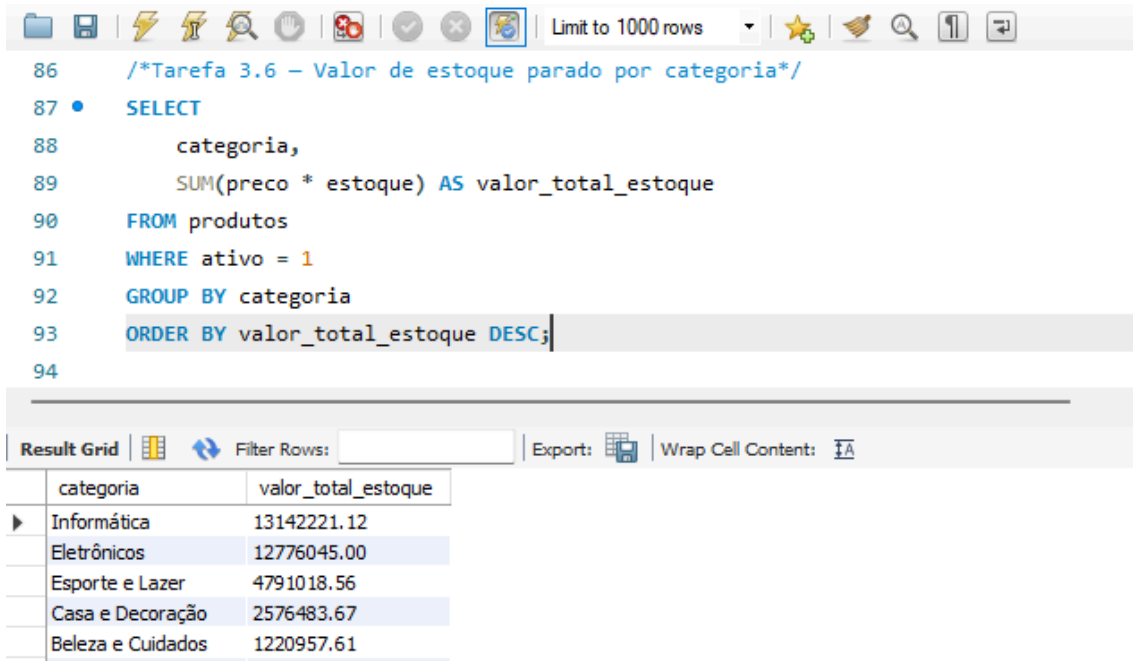
Below the query editor, the 'Result Grid' shows the results of the last query (line 81). The columns are 'segmento' and 'media_idade'.

segmento	media_idade
Varejo	38.40
Corporativo	39.06
Atacado	39.87

Atividade 3.6

Tarefa 3.6 — Valor de estoque parado por categoria Contexto: O financeiro quer saber onde está concentrado o capital investido em estoque. Tarefa: Para produtos ativos, calcule o valor total em estoque (preço × estoque) por categoria, ordenado do maior para o menor.

Evidência esperada: Expressão calculada dentro de uma função agregadora — SUM(preço * estoque) — combinada com GROUP BY e ORDER BY.



The screenshot shows a SQL query editor interface. The query is as follows:

```
86 /*Tarefa 3.6 – Valor de estoque parado por categoria*/
87 • SELECT
88     categoria,
89     SUM(preco * estoque) AS valor_total_estoque
90 FROM produtos
91 WHERE ativo = 1
92 GROUP BY categoria
93 ORDER BY valor_total_estoque DESC;
94
```

Below the query editor is the 'Result Grid' showing the results of the query. The grid has two columns: 'categoria' and 'valor_total_estoque'. The results are ordered from highest to lowest total stock value.

categoria	valor_total_estoque
Informática	13142221.12
Eletrônicos	12776045.00
Esporte e Lazer	4791018.56
Casa e Decoração	2576483.67
Beleza e Cuidados	1220957.61

Atividade 4.1

Bloco 4 — Classificação com CASE e regras de negócio Recursos praticados: CASE, GROUP BY pelo alias do CASE, AVG(CASE WHEN...) para taxas percentuais Números isolados dizem pouco sem uma régua de interpretação. É isso que o CASE resolve. Tarefa 4.1 — Classificando avaliações Tarefa: Usando CASE, classifique cada avaliação (coluna nota) em 'Excelente' (5), 'Boa' (4), 'Regular' (3) ou 'Insatisfatória' (1 ou 2). Evidência esperada: CASE com múltiplas condições e alias para a coluna resultante.

```
/*Tarefa 4.1 – Valor de estoque parado por categoria*/  
SELECT  
    nota,  
    CASE  
        WHEN nota = 5 THEN 'Excelente'  
        WHEN nota = 4 THEN 'Boa'  
        WHEN nota = 3 THEN 'Insatisfatória'  
        WHEN nota = 1 or 2 THEN 'Insatisfatória'  
        ELSE 'Estoque OK'  
    END AS boletin_geral  
FROM avaliacoes;
```

Grid	Filter Rows:	Export:	Wrap Cell Content:
ta	boletin_geral		
	Excelente		
	Excelente		
	Excelente		
	Insatisfatória		
	Insatisfatória		
	Insatisfatória		
	Excelente		
	Insatisfatória		
	Excelente		
	Boa		

Atividade 4.2

Tarefa 4.2 — Quantas avaliações caem em cada faixa Contexto: A qualidade quer um resumo por faixa, não avaliação por avaliação. Tarefa: A partir da classificação da tarefa 4.1, mostre quantas avaliações existem em cada faixa, da maior para a menor quantidade, em uma única consulta. Evidência esperada: Agrupamento pelo alias definido no CASE (GROUP BY na faixa), sem uso de subconsulta.

```
106      /*Tarefa 4.2 – Quantas avaliações caem em cada faixa*/
107      •   SELECT
108      ○     CASE
109              WHEN nota = 5 THEN 'Excelente'
110              WHEN nota = 4 THEN 'Boa'
111              WHEN nota IN (1, 2, 3) THEN 'Insatisfatória'
112              ELSE 'Nota inválida'
113      END AS faixa,
114      COUNT(*) AS total_avaliacoes
115  FROM avaliacoes
116  GROUP BY faixa
117  ORDER BY total_avaliacoes DESC;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	faixa	total_avaliacoes			
▶	Excelente	2402			
	Boa	1826			
	Insatisfatória	1772			

Atividade 4.3

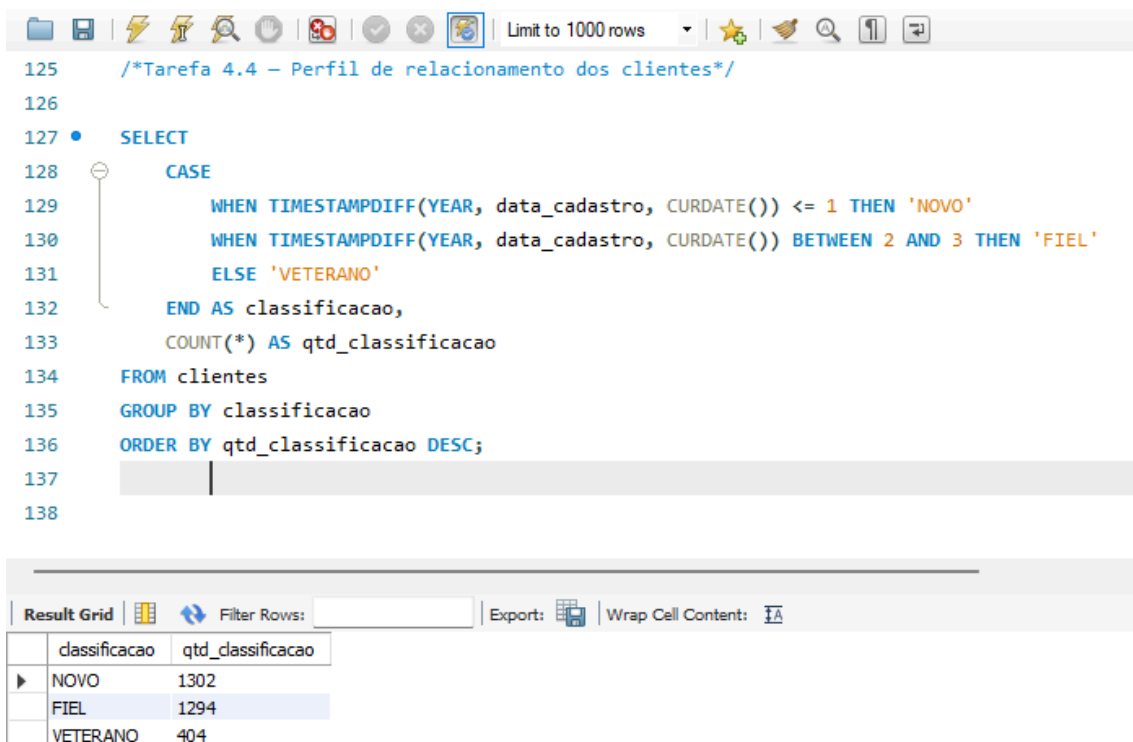
Tarefa 4.3 — Taxa de aprovação de pedidos Contexto: A diretoria pediu, literalmente, "a taxa de aprovação dos pedidos, em percentual" — uma métrica de conversão comum em qualquer negócio digital. Tarefa: Calcule, em uma única consulta, o percentual de pedidos com status = 'Aprovado' em relação ao total de pedidos. Evidência esperada: Uso da técnica $AVG(CASE WHEN \dots THEN 1 ELSE 0 END) * 100$ — uma forma real de calcular taxas sem subconsulta, muito usada em relatórios de mercado.

```
106      /*Tarefa 4.2 – Quantas avaliações caem em cada faixa*/
107      SELECT
108      CASE
109          WHEN nota = 5 THEN 'Excelente'
110          WHEN nota = 4 THEN 'Boa'
111          WHEN nota IN (1, 2, 3) THEN 'Insatisfatória'
112          ELSE 'Nota inválida'
113      END AS faixa,
114      COUNT(*) AS total_avaliacoes
115  FROM avaliacoes
116  GROUP BY faixa
117  ORDER BY total_avaliacoes DESC;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	faixa	total_avaliacoes			
▶	Excelente	2402			
	Boa	1826			
	Insatisfatória	1772			

Atividade 4.4

Tarefa 4.4 — Perfil de relacionamento dos clientes Tarefa: Classifique os clientes em 'Novo' (cadastro há menos de 1 ano), 'Fiel' (entre 1 e 3 anos) ou 'Veterano' (mais de 3 anos), usando CASE combinado com TIMESTAMPDIFF, e mostre quantos clientes existem em cada perfil. Evidência esperada: Função de data usada corretamente dentro do CASE, combinada com GROUP BY



The screenshot displays a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following query:

```
125  /*Tarefa 4.4 – Perfil de relacionamento dos clientes*/
126
127  •  SELECT
128      CASE
129          WHEN TIMESTAMPDIFF(YEAR, data_cadastro, CURDATE()) <= 1 THEN 'NOVO'
130          WHEN TIMESTAMPDIFF(YEAR, data_cadastro, CURDATE()) BETWEEN 2 AND 3 THEN 'FIEL'
131          ELSE 'VETERANO'
132      END AS classificacao,
133      COUNT(*) AS qtd_classificacao
134  FROM clientes
135  GROUP BY classificacao
136  ORDER BY qtd_classificacao DESC;
```

Below the editor, the 'Result Grid' shows the output of the query:

	classificacao	qtd_classificacao
▶	NOVO	1302
	FIEL	1294
	VETERANO	404

Perguntas

1- qual pergunta de negócio a dupla tentou responder e não conseguiu resolver com uma tabela só;

Como está sendo realizado o atendimento das reclamações dos clientes em relação aos pedidos, considerando os diferentes canais de venda?

qual canal de venda possui as melhores avaliações dos clientes?

Em quais canais há mais reclamações?

O setor de suporte está conseguindo responder e atender adequadamente os clientes em cada canal?

(b) em qual tabela está cada informação necessária;

avaliação

pedido

cliente

(c) por que, na opinião da dupla, é necessário combinar as tabelas para responder.

Se observarmos somente os pedidos, por exemplo, conseguimos identificar quantos foram realizados ou cancelados, mas não conseguimos saber como o cliente avaliou o atendimento ou se houve alguma reclamação relacionada àquele pedido. Ao relacionar as informações de pedidos, clientes e avaliações, conseguimos ter uma visão mais completa do processo.

Conclusão geral

Os 3 principais insights encontrados e 1 recomendação de negócio para a NexaShop.

1. Boleto cancela mais

Vimos que quase 3 em cada 10 pessoas que escolhem pagar com boleto desistem da compra. Com Pix ou cartão, isso acontece bem menos (mais ou menos 1 em cada 10). Acharmos que isso acontece porque o boleto demora para confirmar o pagamento, e nesse

tempo o cliente muda de ideia.

```
127  -- -----
128  -- INSIGHT 1 – Boleto cancela quase 3x mais que os outros pagamentos
129  -- (mesma consulta usada na Tarefa 5.3 – taxa de cancelamento por forma
130  -- de pagamento, usando AVG(CASE WHEN...) para calcular percentual)
131  -- -----
132  • SELECT
133      forma_pagamento,
134      COUNT(*) AS total_pedidos,
135      ROUND(
136          AVG(CASE WHEN status = 'Cancelado' THEN 1 ELSE 0 END) * 100,
137          2) AS taxa_cancelamento_percentual
138  FROM pedidos
139  GROUP BY forma_pagamento
140  ORDER BY taxa_cancelamento_percentual DESC;
141
```

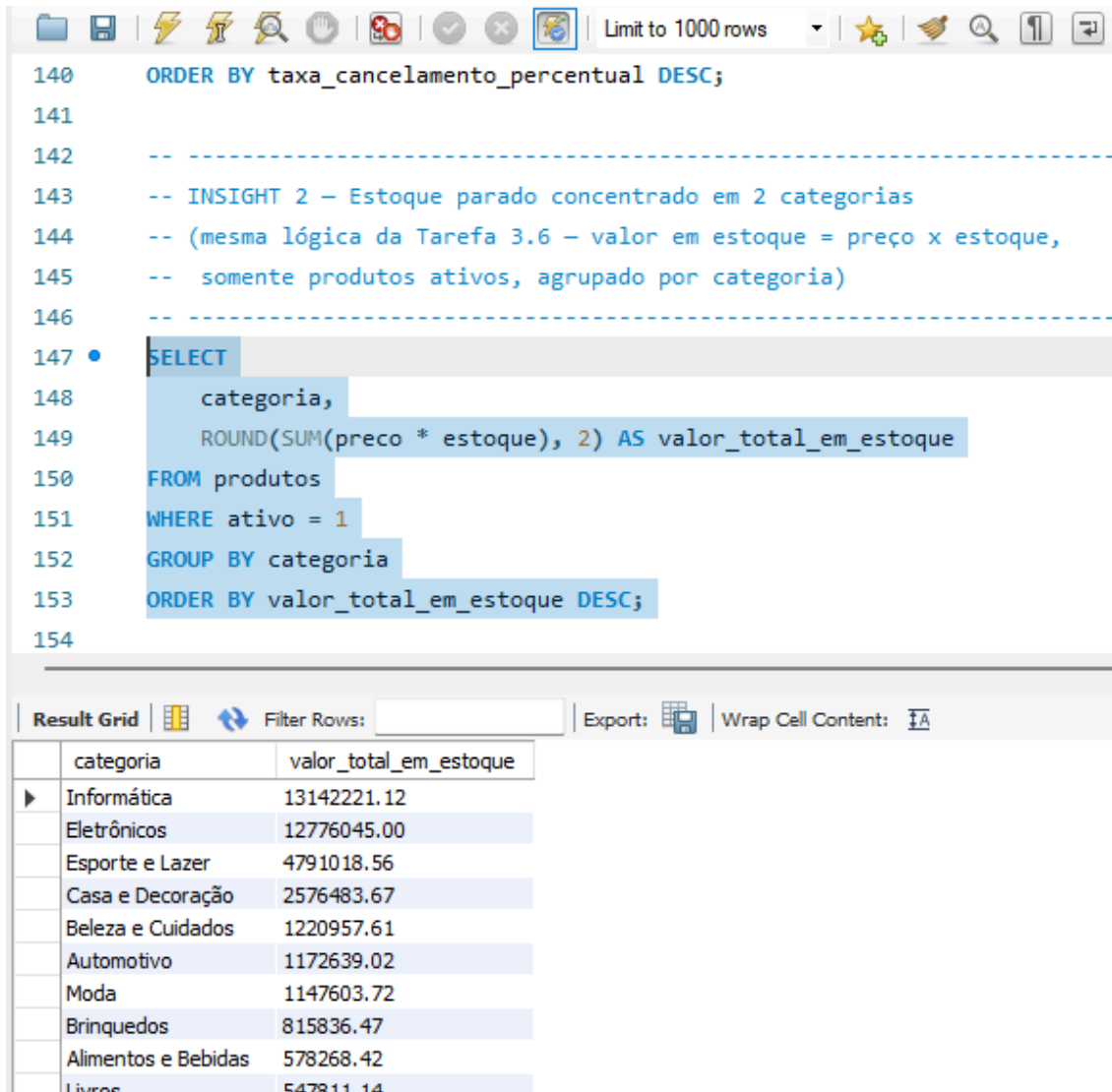
Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	forma_pagamento	total_pedidos	taxa_cancelamento_percentual
▶	Boleto	1397	29.56
	Cartão de Débito	1013	14.22
	Cartão de Crédito	5349	11.95
	Pix	4241	11.74

2. Quase todo o estoque está em só 2 tipos de produto

A loja tem muito dinheiro "parado" em produtos guardados. Quase tudo isso está em só duas categorias: Informática e Eletrônicos. As outras 8 categorias juntas têm bem menos. Achamos isso arriscado, porque se essas duas categorias venderem mal, a loja fica com muito

dinheiro parado.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following code:

```
140 ORDER BY taxa_cancelamento_percentual DESC;
141
142 -----
143 -- INSIGHT 2 – Estoque parado concentrado em 2 categorias
144 -- (mesma lógica da Tarefa 3.6 – valor em estoque = preço x estoque,
145 -- somente produtos ativos, agrupado por categoria)
146 -----
147 • SELECT
148     categoria,
149     ROUND(SUM(preço * estoque), 2) AS valor_total_em_estoque
150 FROM produtos
151 WHERE ativo = 1
152 GROUP BY categoria
153 ORDER BY valor_total_em_estoque DESC;
154
```

Below the editor is the 'Result Grid' section, which includes a 'Filter Rows' input and 'Export' and 'Wrap Cell Content' options. The results are displayed in a table:

categoria	valor_total_em_estoque
Informática	13142221.12
Eletrônicos	12776045.00
Esporte e Lazer	4791018.56
Casa e Decoração	2576483.67
Beleza e Cuidados	1220957.61
Automotivo	1172639.02
Moda	1147603.72
Brinquedos	815836.47
Alimentos e Bebidas	578268.42
Livros	547811.14

3. A loja perde quase 3 em cada 10 vendas

De cada 10 pedidos, só 7 viram venda de verdade. Os outros 3 se perdem: são cancelados, ficam parados ou são devolvidos. E, como vimos no primeiro ponto, o boleto é

um dos grandes culpados por essa perda.

```
155  -----
156  -- INSIGHT 3 – Quase 3 em cada 10 pedidos não viram venda aprovada
157  -- (mesma lógica da Tarefa 4.3 – taxa de aprovação com AVG(CASE...),
158  -- complementada por uma contagem simples de status para mostrar
159  -- onde exatamente os pedidos "se perdem")
160  -----
161  • SELECT
162      ROUND(
163          AVG(CASE WHEN status = 'Aprovado' THEN 1 ELSE 0 END) * 100,
164          2) AS taxa_aprovacao_percentual
165  FROM pedidos;
166
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
taxa_aprovacao_percentual				
▶	70.40			

Nossa recomendação:

Como o boleto cancela muito mais, achamos que a NexaShop deveria dar um desconto para quem pagar com Pix. Assim, mais gente escolhe o Pix (que confirma na hora) em vez do boleto (que demora e cancela mais), e a loja perde menos vendas.